

QuickChat — Bluetooth Messaging Application

Protocol Design Document

AUTHOR
Niha Hawas

COURSE
CNDC — Computer Networks & Data Communications

PROGRAM
BSCS-6B, SZABIST Islamabad

DATE
2026

1. What Is This Document?

This document describes the communication protocol used by QuickChat — the rules both Android devices follow when exchanging text messages and files over Bluetooth. It defines exactly how data is structured at the byte level so both the sender and receiver always understand each other.

Simple analogy: Think of sending a letter in an envelope. The envelope always has a label showing what's inside and how heavy it is. Our protocol works the same way — every piece of data is wrapped in a small header that tells the receiver what type of data it is and how many bytes to expect.

Property	Value	What it means
Connection Type	Bluetooth Classic — RFCOMM	Acts like a wireless serial cable between the two phones
Service UUID	00001101-0000-1000-8000-00805F9B34FB	Unique ID that identifies QuickChat's Bluetooth service (SPP profile)
Text Encoding	UTF-8	Standard encoding that supports all languages and characters
Byte Order	Big-Endian	Most significant byte is sent first (standard network order)
Max File Chunk	4,096 bytes (4 KB)	Large files are broken into 4 KB pieces for reliable transfer

2. Packet Structure — The Universal Envelope

Every transmission — whether a text message, a file chunk, or a control signal — uses the same **packet format**. A packet has two parts: a fixed 5-byte **header** that describes the data, and a variable-length **payload** that contains the actual data.

TYPE	LENGTH	PAYLOAD
1 byte	4 bytes (uint32)	LENGTH bytes (variable)
What kind of data?	How many bytes in payload?	The actual message, file data, or command

How the receiver works: It always reads exactly 5 bytes first. Byte 1 tells it the type. Bytes 2–5 tell it exactly how many more bytes to read. This ensures the receiver never reads too much or too little — data stays perfectly in sync.

2.1 Packet Type Codes

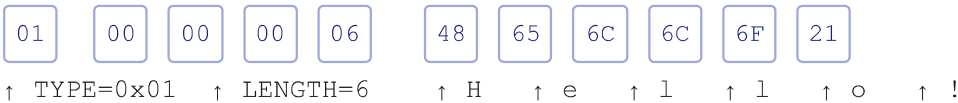
Code	Name	Plain English — What it does
0x01	TEXT_MESSAGE	A chat message typed by the user
0x02	FILE_START	"I am about to send you a file — here are its details"
0x03	FILE_CHUNK	One 4 KB piece of the file being transferred
0x04	FILE_END	"That was the last piece — here is a checksum to verify the file"
0x05	ACK	"I received your message or file" (success or failure)
0x06	PING	"Are you still connected?" — keeps the connection alive

3. Text Message Packet (Type 0x01)

When a user types a message and taps Send, the app builds a TEXT_MESSAGE packet. The payload is simply the message text converted to UTF-8 bytes — nothing more.

Example — Sending the message "Hello!"

Full packet bytes:



Field	Value
TYPE	0x01
LENGTH	Number of UTF-8 bytes in the message (e.g., 6 for "Hello!")
PAYLOAD	The message text encoded as UTF-8 bytes

4. File Transfer (Types 0x02, 0x03, 0x04)

Transferring a file happens in **three clearly defined stages**. This lets the receiver know what is coming before it arrives, track progress with a real percentage, and verify the file is complete and undamaged at the end.

Why split files into 4 KB chunks? Bluetooth has limited and variable bandwidth. Sending one large block of data is risky — a single glitch corrupts everything. Chunks allow the progress bar to update smoothly and make error detection much easier.

Stage 1 — FILE_START (0x02): Announce the file

Sent once before any file data. The payload is a JSON object so the receiver knows the filename, total size, type, and how many chunks to expect.

```
Payload (JSON, UTF-8 encoded):  
{ "name": "photo.jpg", "size": 204800, "mime": "image/jpeg", "chunks": 50 }
```

JSON Field	Type	Meaning & Example
name	String	Original filename — e.g., "photo.jpg"
size	Integer	Total file size in bytes — e.g., 204800 = 200 KB
mime	String	File type — e.g., "image/jpeg" or "application/pdf"
chunks	Integer	Number of FILE_CHUNK packets that will follow — e.g., 50

Stage 2 — FILE_CHUNK (0x03): Send the file data

The sender reads the file 4 KB at a time and sends each piece as a FILE_CHUNK packet. The receiver appends each chunk in order to reconstruct the file. After each chunk is received, the progress bar updates: $progress = (chunks\ received \div total\ chunks) \times 100\%$

```
Example — Chunk 1 of a 200 KB file (50 chunks total):  
  
03 00 00 10 00 [4096 bytes of raw file data ...]  
↑ TYPE=chunk   ↑ LENGTH = 4096 bytes
```

Stage 3 — FILE_END (0x04): Verify completion

After the last chunk, the sender transmits FILE_END. The payload is a 32-character **MD5 checksum** — a fingerprint of the entire file. The receiver recalculates the MD5 from the assembled file and

compares. If they match, the file is perfect. If not, it sends a failure ACK.

04

00

00

00

20

d41d8cd98f00b204e9800998ecf8427e

↑ TYPE=end

↑ LENGTH=32

↑ MD5 checksum (32 ASCII characters)

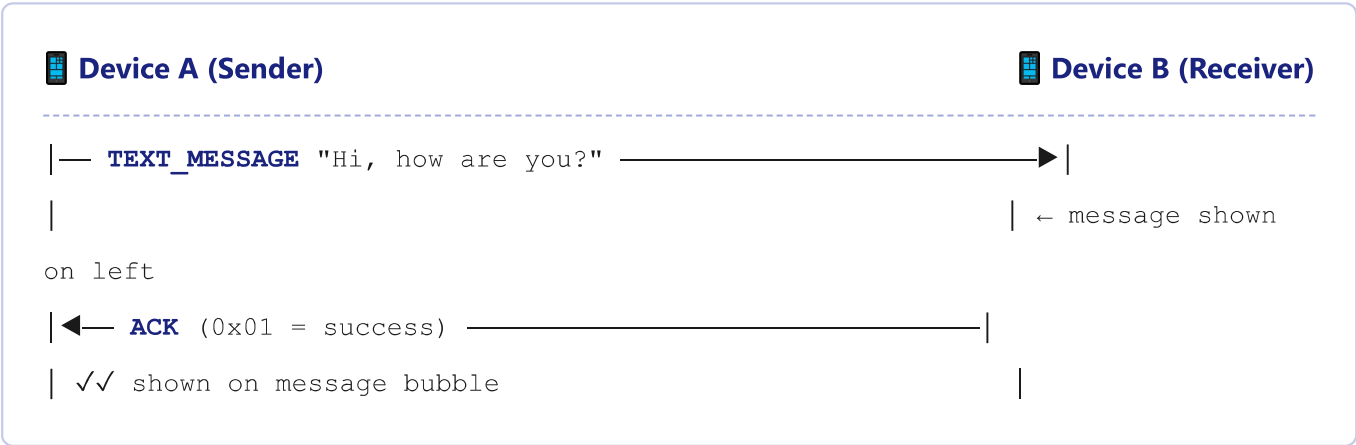
5. Acknowledgement Packet — ACK (0x05)

After successfully receiving a text message or a complete file, the receiver sends back an ACK. This is what powers the **delivery receipt (✓✓) feature** — the sender only marks a message as delivered once it receives this ACK.

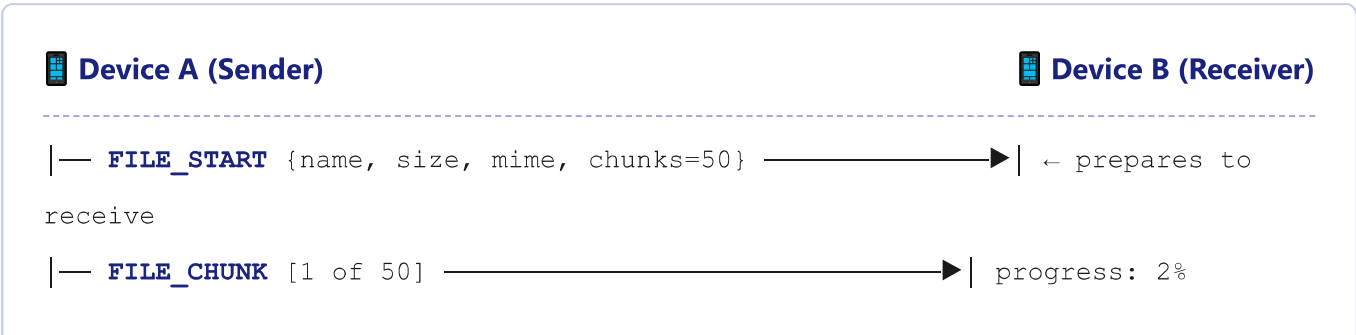
ACK Payload Byte	Meaning	What the sender's UI shows
0x01	Success — data received correctly	Double tick ✓✓ appears on the message bubble
0x00	Failure — checksum mismatch or buffer error	Error icon shown, retry button offered

6. Communication Flow — Step by Step

6.1 Sending a Text Message



6.2 Sending a File



```

|— FILE_CHUNK [2 of 50] —————▶| progress: 4%
|— ... (chunks 3 through 49) ... —————▶| progress growing...
|— FILE_CHUNK [50 of 50] —————▶| progress: 100%
|— FILE_END (MD5 checksum) —————▶| ← verifies checksum
|◀— ACK (0x01 success / 0x00 fail) —————|
| "File sent ✓" or retry error                                | ← file preview
shown

```

7. Error Handling

- **Checksum mismatch:** If the receiver's MD5 does not match the FILE_END value, it sends ACK 0x00 . The sender displays an error message and offers a retry button.
- **Connection drop mid-transfer:** If the Bluetooth socket closes unexpectedly, both devices clear their receive buffers and display a "Disconnected" banner. The user can reconnect and retry the transfer.
- **Keep-alive timeout:** If no data arrives for 30 seconds, a PING packet is sent. If no response comes within 5 seconds, the connection is declared lost and the UI updates accordingly.
- **Sequential transfers only:** Only one file can be sent at a time. Any text messages sent during a file transfer are queued locally and delivered automatically once the transfer completes.

Design note: The protocol is intentionally simple and sequential — there is no packet numbering or automatic retransmission. Reliability comes from the RFCOMM transport layer and the MD5 integrity check at the end of each file transfer. This keeps the implementation straightforward while still being robust for typical use cases.